

Operating Systems

Assignment 1

Sambit Sahoo (22277)

*My username is **sambit** (You can find it in all screenshots.)*

Q1 - With this problem, we will understand the status of a running process using the top command. To do this, I compiled the program cpu.c and ran it on the Bash terminal as follows:

```
gcc cpu.c -o cpu
./cpu
```

This program executes an infinite loop. Now I opened another terminal window and ran the **top** command. Then I observed the following about the **cpu** process:

- (a) Determine the PID of the process executing the **cpu** program.

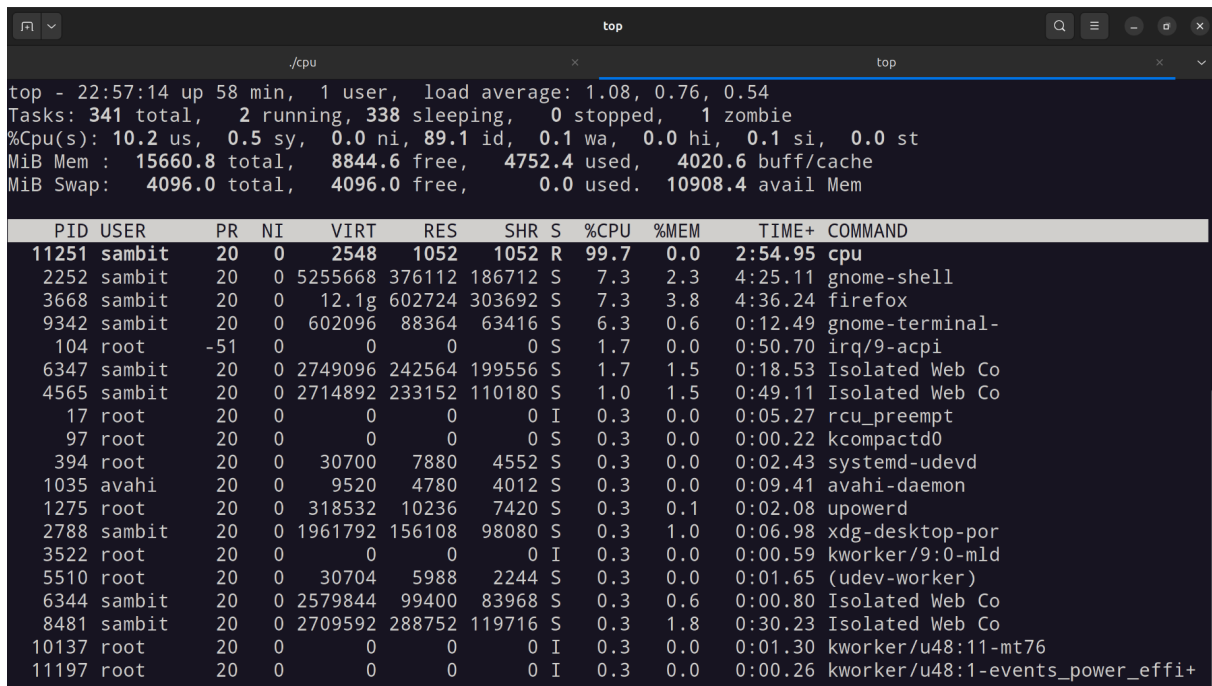
Answer: 11251

- (b) How much memory and CPU does this process consume?

Answer: About 1 MB of physical memory (RES = 1052 KB), with a total virtual memory of 2.5 MB (VIRT = 2548 KB) so **0.0% Mem** and **99.7% CPU** usage (about 1 full core)

- (c) Determine the current status of the process. For instance, is it executing in a blocked, active, or zombie state?

Answer: The **R** in the column S as shown in the figure below denotes that the process is in **Running State**.



```
top - 22:57:14 up 58 min, 1 user, load average: 1.08, 0.76, 0.54
Tasks: 341 total, 2 running, 338 sleeping, 0 stopped, 1 zombie
%Cpu(s): 10.2 us, 0.5 sy, 0.0 ni, 89.1 id, 0.1 wa, 0.0 hi, 0.1 si, 0.0 st
MiB Mem : 15660.8 total, 8844.6 free, 4752.4 used, 4020.6 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used, 10908.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
11251	sambit	20	0	2548	1052	1052	R	99.7	0.0	2:54.95	cpu
2252	sambit	20	0	5255668	376112	186712	S	7.3	2.3	4:25.11	gnome-shell
3668	sambit	20	0	12.1g	602724	303692	S	7.3	3.8	4:36.24	firefox
9342	sambit	20	0	602096	88364	63416	S	6.3	0.6	0:12.49	gnome-terminal-
104	root	-51	0	0	0	0	S	1.7	0.0	0:50.70	irq/9-acpi
6347	sambit	20	0	2749096	242564	199556	S	1.7	1.5	0:18.53	Isolated Web Co
4565	sambit	20	0	2714892	233152	110180	S	1.0	1.5	0:49.11	Isolated Web Co
17	root	20	0	0	0	0	I	0.3	0.0	0:05.27	rcu_preempt
97	root	20	0	0	0	0	S	0.3	0.0	0:00.22	kcompactd0
394	root	20	0	30700	7880	4552	S	0.3	0.0	0:02.43	systemd-udev
1035	avahi	20	0	9520	4780	4012	S	0.3	0.0	0:09.41	avahi-daemon
1275	root	20	0	318532	10236	7420	S	0.3	0.1	0:02.08	upowerd
2788	sambit	20	0	1961792	156108	98080	S	0.3	1.0	0:06.98	xdg-desktop-por
3522	root	20	0	0	0	0	I	0.3	0.0	0:00.59	kworker/9:0-mld
5510	root	20	0	30704	5988	2244	S	0.3	0.0	0:01.65	(udev-worker)
6344	sambit	20	0	2579844	99400	83968	S	0.3	0.6	0:00.80	Isolated Web Co
8481	sambit	20	0	2709592	288752	119716	S	0.3	1.8	0:30.23	Isolated Web Co
10137	root	20	0	0	0	0	I	0.3	0.0	0:01.30	kworker/u48:11-mt76
11197	root	20	0	0	0	0	I	0.3	0.0	0:00.26	kworker/u48:1-events_power_effi+

Q2 - In this problem, we understand the hardware configuration of a Linux machine using the /proc filesystem and the lscpu command. First, I studied CPU hyperthreading at a high level. Then I executed the following commands:

```
more /proc/cpuinfo
more /proc/meminfo
cat /proc/stats
lscpu
```

- **Hyperthreading:** It means one physical core can expose multiple hardware threads so, OS sees multiple processors (logical CPUs) per core.
- **Cores:** Physical processing units inside a CPU that execute instructions.
- **Processors:** A processing unit recognized by the operating system; may be a physical core or a virtual core created using hyperthreading.

(a) How many cores does your machine have?

Answer: 10

(b) How many processors (logical CPUs) does your machine have?

Answer: 12, because I have 2 performance cores and 8 efficiency cores and each P-core has 2 threads and each E-core has 1 thread. (So $2*2 + 1*8 = 12$).

(c) Determine the frequency of each processor.

Answer:

- | | |
|------------------------------|--------------------------|
| • Processor 0 – 1037.305 MHz | • Processor 6 – 400 MHz |
| • Processor 1 – 400 MHz | • Processor 7 – 400 MHz |
| • Processor 2 – 1085.467 MHz | • Processor 8 – 400 MHz |
| • Processor 3 – 400 MHz | • Processor 9 – 400 MHz |
| • Processor 4 – 569.607 MHz | • Processor 10 – 400 MHz |
| • Processor 5 – 521.591 MHz | • Processor 11 – 400 MHz |

(d) Determine how much physical memory your system has.

Answer: 16036700 KB (15.3 GB)

(e) Determine the architecture of the CPU.

Answer: x86_64

(f) How much of the memory is currently occupied?

Answer: 5720304 kB = 5.46 GB

(g) Determine the total number of forks and context switches since system boot.

Answer: Forks = 7730, Context Switches = 13670002

```
sambit@sambit-PC:~$ more /proc/cpuinfo
processor          : 0
vendor_id         : GenuineIntel
cpu family        : 6
model             : 154
model name        : 12th Gen Intel(R) Core(TM) i5-1235U
stepping          : 4
microcode         : 0x436
cpu MHz           : 1037.305
cache size        : 12288 KB
physical id       : 0
siblings          : 12
core id           : 0
cpu cores         : 10
apicid            : 0
initial apicid    : 0
fpu               : yes
fpu_exception     : yes
cpuid level       : 32
wp               : yes
```

```
sambit@sambit-PC:~$ more /proc/meminfo
MemTotal:          16036700 kB
MemFree:           10316396 kB
MemAvailable:      12164068 kB
```

```
sambit@sambit-PC:~$ lscpu
Architecture:      x86_64
CPU op-mode(s):    32-bit, 64-bit
Address sizes:      39 bits physical, 48 bits virtual
Byte Order:        Little Endian
```

```
0 0 0 0 0 0 0 0
ctxt 13670002
btime 1769274925
processes 7730
procs_running 2
procs_blocked 1
```

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ lscpu -e=CPU,CORE
CPU CORE
0 0
1 0
2 1
3 1
4 2
5 3
6 4
7 5
8 6
9 7
10 8
11 9
```

Q3- I executed the two programs memory_1.c and memory_2.c given. The following are my observations:

Both of the files memory_1.c and memory_2.c allocate array of size `ARRAY_SIZE = 1000000` integers, so the **virtual memory (VSZ)** output given by the `ps` command is almost same (about 6.3MB) for both program. The **physical memory (RSS)** is also similar between the two programs.(**5276kB** in memory_1 and **5404kB** in memory_2)

Memory usage of both programs appears almost same because in both cases the array is allocated on the **stack**. Even though memory_1 never accesses the array, the stack space for it is still reserved in the process's address space, since stack allocation in C is done upfront.

If array was instead been allocated on the **heap**, the behavior would have been different. In that case, the OS uses **demand paging**, meaning memory pages are brought to physical RAM only when they are actually accessed. So, if the memory_1 had allocated the array dynamically but has never touched it, its RSS would be much smaller, showing that those pages were not loaded into RAM.

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ ./memory_1

Program : 'memory_1'
-----

PID : 14978
Size of int : 4

Press Enter Key to exit.

sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ ./memory_2

Program : 'memory_2'
-----

PID : 15018
Size of int : 4

Press Enter Key to exit.
```

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ ps -o pid,vsz,sz,rss,cmd 14978
  PID   VSZ    SZ    RSS CMD
 14978  6464  1616   5276 ./memory_1
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ ps -o pid,vsz,sz,rss,cmd 15018
  PID   VSZ    SZ    RSS CMD
 15018  6468  1617   5404 ./memory_2
```

Q4(a) - I compiled the program `cpu-print.c` and executed it in the bash by using

```
gcc cpu-print.c -o cpu-print
./cpu-print
```

Then I opened another terminal and use the `ps -C cpu-print` to find out the pid of the process spawned by the shell to run the `cpu-print` executable as shown in the picture below.

```
sambit@sambit-PC:~$ ps -C cpu-print
  PID TTY          TIME CMD
 16703 pts/0    00:01:46 cpu-print
sambit@sambit-PC:~$ ps -F 16703
UID          PID    PPID  C   SZ   RSS PSR STIME TTY          STAT       TIME CMD
sambit      16703    6058  85   670  1308   0 17:19 pts/0    S+          2:31 ./cpu-print
sambit@sambit-PC:~$ ps -F 6058
UID          PID    PPID  C   SZ   RSS PSR STIME TTY          STAT       TIME CMD
sambit       6058    6018   0  2849  5416  10 Jan24 pts/0    S           0:00 bash
sambit@sambit-PC:~$ ps -F 6018
UID          PID    PPID  C   SZ   RSS PSR STIME TTY          STAT       TIME CMD
sambit       6018    6011   0  3280  6888  10 Jan24 pts/0    Ss          0:00 zsh
sambit@sambit-PC:~$ ps -F 6011
UID          PID    PPID  C   SZ   RSS PSR STIME TTY          STAT       TIME CMD
sambit       6011    2238   0 185555 88196   2 Jan24 ?        Dsl         4:17 /usr/libexec/gnome-terminal-serve
sambit@sambit-PC:~$ ps -F 2238
UID          PID    PPID  C   SZ   RSS PSR STIME TTY          STAT       TIME CMD
sambit       2238       1   0  5358 13536   6 Jan24 ?        Ss          0:03 /usr/lib/systemd/systemd --user
sambit@sambit-PC:~$ ps -F 1
UID          PID    PPID  C   SZ   RSS PSR STIME TTY          STAT       TIME CMD
root          1         0   0  5917 14136   3 Jan24 ?        Ss          0:05 /sbin/init splash
```

Q4(b) - I found the PID of the parent of the `cpu-print` process using "`ps -F (child process id)`". Next, I found the PIDs of all the ancestors, going back until I reached the limit process (i.e `PID = 1`)) as we can see from the picture above.

Q4(c) - I then understood how the shell performs output redirection by executing the following command. `./cpu-print > /tmp/tmp.txt`.

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ ./cpu-print > /tmp/tmp.txt &
[1] 17211
```

```
sambit@sambit-PC:~$ stat /proc/17211/fd/0
  File: /proc/17211/fd/0 -> /dev/pts/0
Size: 64      Blocks: 0          IO Block: 1024   symbolic link
Device: 0,25  Inode: 132491     Links: 1
Access: (0700/lrwx-----)  Uid: ( 1000/  sambit)   Gid: ( 1000/  sambit)
Access: 2026-01-25 17:32:33.508822845 +0530
Modify: 2026-01-25 17:32:33.508822845 +0530
Change: 2026-01-25 17:32:33.508822845 +0530
Birth: -
sambit@sambit-PC:~$ stat /proc/17211/fd/1
  File: /proc/17211/fd/1 -> /tmp/tmp.txt
Size: 64      Blocks: 0          IO Block: 1024   symbolic link
Device: 0,25  Inode: 132496     Links: 1
Access: (0300/l-wx-----)  Uid: ( 1000/  sambit)   Gid: ( 1000/  sambit)
Access: 2026-01-25 17:32:35.870967498 +0530
Modify: 2026-01-25 17:32:35.870967498 +0530
Change: 2026-01-25 17:32:35.870967498 +0530
Birth: -
sambit@sambit-PC:~$ stat /proc/17211/fd/2
  File: /proc/17211/fd/2 -> /dev/pts/0
Size: 64      Blocks: 0          IO Block: 1024   symbolic link
Device: 0,25  Inode: 132501     Links: 1
Access: (0700/lrwx-----)  Uid: ( 1000/  sambit)   Gid: ( 1000/  sambit)
Access: 2026-01-25 17:32:37.536069071 +0530
Modify: 2026-01-25 17:32:37.536069071 +0530
Change: 2026-01-25 17:32:37.536069071 +0530
Birth: -
```

As we can see the above picture, the file descriptors of the process show that:

- FD 0 (stdin) points to **/dev/pts/0** (keyboard/terminal input).
- FD 1 (stdout) points to **/tmp/tmp.txt** (output redirected to a file).
- FD 2 (stderr) points to **/dev/pts/0** (terminal error output).

So shell implements I/O redirection by changing where file descriptors 0, 1, and 2 point before executing the program. By redirecting these descriptors to files or devices, the shell controls where a program reads input from and where it writes its output and error messages.

Q4(d) - I executed the following command `./cpu-print | grep hello &`. I observed the following:

When running the command `./cpu-print — grep hello &`, the shell creates a pipe and connects the standard output of `cpu-print` to the standard input of `grep`. From the `/proc/<pid>/fd` information:

- For `cpu-print`, its FD 1 (stdout) points to a pipe.
- For `grep`, its FD 0 (stdin) points to the same pipe.
- FD 2 (stderr) of both processes still points to the terminal (**/dev/pts/0**).

This shows that the shell implements pipes by creating a pipe object in the kernel and then redirecting the file descriptors so that the output of one process becomes the input of the next process through the pipe.

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ ./cpu-print | grep hello &
[1] 4474
```

```
sambit@sambit-PC:~$ stat /proc/4474/fd/0
  File: /proc/4474/fd/0 -> pipe:[37216]
  Size: 64          Blocks: 0          IO Block: 1024   symbolic link
Device: 0,25   Inode: 22499          Links: 1
Access: (0500/lr-x-----)  Uid: ( 1000/  sambit)   Gid: ( 1000/  sambit)
Access: 2026-01-25 17:41:48.806760497 +0530
Modify: 2026-01-25 17:41:48.806760497 +0530
Change: 2026-01-25 17:41:48.806760497 +0530
Birth: -
sambit@sambit-PC:~$ stat /proc/4474/fd/1
  File: /proc/4474/fd/1 -> /dev/pts/0
  Size: 64          Blocks: 0          IO Block: 1024   symbolic link
Device: 0,25   Inode: 35940          Links: 1
Access: (0700/lrwx-----)  Uid: ( 1000/  sambit)   Gid: ( 1000/  sambit)
Access: 2026-01-25 17:41:51.986549452 +0530
Modify: 2026-01-25 17:41:51.986549452 +0530
Change: 2026-01-25 17:41:51.986549452 +0530
Birth: -
sambit@sambit-PC:~$ stat /proc/4474/fd/2
  File: /proc/4474/fd/2 -> /dev/pts/0
  Size: 64          Blocks: 0          IO Block: 1024   symbolic link
Device: 0,25   Inode: 37224          Links: 1
Access: (0700/lrwx-----)  Uid: ( 1000/  sambit)   Gid: ( 1000/  sambit)
Access: 2026-01-25 17:41:55.212959605 +0530
Modify: 2026-01-25 17:41:55.212959605 +0530
Change: 2026-01-25 17:41:55.212959605 +0530
Birth: -
```

Q4(e) - I typed the commands `cd`, `ls`, `history`, and `ps` into my bash shell. From the output of the type command:

- `cd` and `history` are shell built-in commands implemented inside the Bash shell.
- `ls` and `ps` are external executables located in the Linux filesystem (e.g., `/bin/ls` and `/usr/bin/ps`) and are invoked by the shell as separate programs.

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ type cd
cd is a shell builtin
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ type ls
ls is aliased to `ls --color=auto'
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ type history
history is a shell builtin
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ type ps
ps is /usr/bin/ps
```

Q5 - I used the command `iostat -dh nvme0n1 5` after running the `./disk_1` & `./disk_2` in separate terminals and My Observations for this question are as follows:

- Both `disk_1.c` and `disk_2.c` repeatedly reads files from the `disk-files` directory, and their disk activities is observed using the `iostat` command.
- In `disk_1.c`, a random file is selected in each iteration (`file_no = rand() % MAX_FILE_NO`). This causes frequent disk seeks and scattered read operations. As a result, `iostat` reports high transactions per second (tps) and significant disk read throughput, indicating a heavy disk utilization. (i.e, first image below)
- In `disk_2.c`, the same file is read repeatedly (`file_no = 1`). After the first read, the file is being cached in the OS disk buffer cache. Subsequent reads are served from memory instead of the physical disk. So, `iostat` shows very low or nearly zero disk read activity after the initial access. (i.e, second image below)

```
sambit@sambit-PC:~$ iostat -dh nvme0n1 5
Linux 6.11.0-19-generic (sambit-PC)      01/25/2026      _x86_64_      (12 CPU)

   tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd Device
   10.57      132.7k      396.7k        0.0k      2.5G      7.4G        0.0k nvme0n1

   tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd Device
    0.60         0.0k        9.6k        0.0k        0.0k      48.0k        0.0k nvme0n1

   tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd Device
  1793.80     151.9M        7.2k        0.0k     759.3M      36.0k        0.0k nvme0n1

   tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd Device
  1221.00     102.6M     266.4k        0.0k     513.2M      1.3M        0.0k nvme0n1

   tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd Device
   55.60       3.7M     308.0k        0.0k      18.3M      1.5M        0.0k nvme0n1

   tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd Device
   34.20       3.0M      76.8k        0.0k      14.8M     384.0k        0.0k nvme0n1
```



```
sambit@sambit-PC: ~  
sambit@sambit-PC: ~$ iostat -dhy nvme0n1 1  
Linux 6.11.0-19-generic (sambit-PC) 01/26/2026 _x86_64_ (12 CPU)  
  
tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd    Device  
0.00    0.0k          0.0k        0.0k        0.0k        0.0k        0.0k    nvme0n1  
  
tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd    Device  
0.00    0.0k          0.0k        0.0k        0.0k        0.0k        0.0k    nvme0n1  
  
tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd    Device  
38.00    1.5M          0.0k        0.0k        1.5M        0.0k        0.0k    nvme0n1  
  
tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd    Device  
0.00    0.0k          0.0k        0.0k        0.0k        0.0k        0.0k    nvme0n1  
  
tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd    Device  
0.00    0.0k          0.0k        0.0k        0.0k        0.0k        0.0k    nvme0n1  
  
tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd    Device  
0.00    0.0k          0.0k        0.0k        0.0k        0.0k        0.0k    nvme0n1
```

Q6 - After executing the commands `strace -c ps` and `strace -c ./memory_1`, I observed the system calls made by the `ps` and `memory_1` executable file from this assignment command as shown below in the pictures:

Total no. of syscalls by:

- `ps`: 4403
- `memory_1`: 44

The `ps` command displays information about running processes. From the `strace -c ps` output, it can be observed that it performs the following system calls:

- `openat` to open files in the `/proc` directory (such as `/proc/stat`, `/proc/[pid]/status`, etc.).
- `read` to read process information from these files.
- `getdents64` to scan the `/proc` directory and obtain the list of process IDs.
- `fstat` and `newfstatat` to retrieve file and process metadata.
- `mmap` and `brk` to allocate memory for storing process information.
- `execve` to execute the `ps` program.
- `write` to output the formatted process information to the terminal.


```
sambit@sambit-PC:~$ strace -c ps
```

PID	TTY	TIME	CMD
4163	pts/1	00:00:00	zsh
4195	pts/1	00:00:00	bash
5411	pts/1	00:00:00	strace
5414	pts/1	00:00:00	ps

% time	seconds	usecs/call	calls	errors	syscall
34.97	0.007809	4	1597	550	openat
25.58	0.005713	4	1268		read
12.61	0.002816	2	1049		close
8.59	0.001918	1918	1		execve
6.58	0.001469	26	56		mmap
5.06	0.001130	3	333	12	newfstatat
2.60	0.000581	145	4		getdents64
1.04	0.000232	17	13		mprotect
0.79	0.000177	7	24		fstat
0.51	0.000115	28	4		munmap
0.50	0.000112	5	21		rt_sigaction
0.46	0.000102	14	7		brk
0.16	0.000035	5	6	4	prctl
0.10	0.000023	11	2		pread64
0.09	0.000021	4	5		write
0.04	0.000010	5	2		ioctl
0.04	0.000009	9	1		prlimit64
0.04	0.000009	9	1		getrandom
0.04	0.000008	8	1		getpid
0.04	0.000008	8	1		arch_prctl
0.03	0.000007	7	1		sched_getaffinity
0.03	0.000007	7	1		set_tid_address
0.03	0.000007	7	1		set_robust_list
0.03	0.000007	7	1		rseq
0.02	0.000004	4	1		geteuid
0.01	0.000003	3	1		futex
0.00	0.000000	0	1	1	access
100.00	0.022332	5	4403	567	total

```
sambit@sambit-PC:~/Documents/SEM 8/OS/Assignment_1$ strace -c ./memory_1
```

Program : 'memory_1'

PID : 5539
Size of int : 4

Press Enter Key to exit.

% time	seconds	usecs/call	calls	errors	syscall
46.75	0.000504	504	1		execve
14.10	0.000152	19	8		mmap
7.88	0.000085	10	8		write
7.24	0.000078	39	2		read
5.47	0.000059	19	3		mprotect
3.06	0.000033	8	4		fstat
2.97	0.000032	10	3		brk
2.78	0.000030	15	2		openat
2.32	0.000025	25	1		munmap
1.21	0.000013	13	1	1	access
1.11	0.000012	6	2		pread64
1.02	0.000011	5	2		close
0.93	0.000010	10	1		getrandom
0.65	0.000007	7	1		prlimit64
0.56	0.000006	6	1		getpid
0.56	0.000006	6	1		arch_prctl
0.46	0.000005	5	1		set_tid_address
0.46	0.000005	5	1		set_robust_list
0.46	0.000005	5	1		rseq
100.00	0.001078	24	44	1	total